

SQLite in Flutter – Complete Documentation

1 . Introduction

SQLite is a Flutter plugin used to work with SQLite databases. It allows you to store data device.

SQLite is a lightweight, local database that works offline.

2 . Add Dependency

Add this to your `pubspec.yaml` file:

```
dependencies:  
  sqflite: ^2.3.0  
  path: ^1.8.3
```

Then run:

```
flutter pub get
```

3 . Import Packages

```
import 'package:sqflite/sqflite.dart';  
import 'package:path/path.dart';
```

4 . Create Database Helper Class

Create a file called `database_helper.dart`.

```
class DatabaseHelper {  
  static Database? _database;  
  
  Future<Database> get database async {  
    if (_database != null) return _database!;
```

```

        _database = await initDatabase();
        return _database!;
    }

    Future<Database> initDatabase() async {
        String path = join(await getDatabasesPath(), 'example.db');

        return await openDatabase(
            path,
            version: 1,
            onCreate: (db, version) async {
                await db.execute('''
                    CREATE TABLE users(
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        name TEXT,
                        age INTEGER
                    )
                ''');
            },
        );
    }
}

```

5 . Insert Data

```

Future<int> insertUser(Map<String, dynamic> user) async {
    final db = await database;
    return await db.insert('users', user);
}

```

Example:

```

await insertUser({
    'name': 'Rasel',
    'age': 25,
});

```

6 . Fetch Data

```
Future<List<Map<String, dynamic>>> getUsers() async {  
  final db = await database;  
  return await db.query('users');  
}
```

7 . Update Data

```
Future<int> updateUser(int id, Map<String, dynamic> user) async {  
  final db = await database;  
  return await db.update(  
    'users',  
    user,  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}
```

8 . Delete Data

```
Future<int> deleteUser(int id) async {  
  final db = await database;  
  return await db.delete(  
    'users',  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}
```

9 . Close Database

```
Future close() async {  
  final db = await database;  
  db.close();  
}
```

1 0 . Best Practices

- Use a single DatabaseHelper instance (Singleton pattern)
 - Keep database logic separate from UI
 - Use models instead of raw Map when possible
 - Always handle exceptions with try-catch
 - Use async/await properly
-

1 1 . Folder Structure Example

```
lib/  
├── data/  
│   ├── database_helper.dart  
│   ├── user_model.dart  
│   └── user_repository.dart  
├── presentation/  
│   └── screens/
```

1 2 . Summary

SQLite helps you:

✓ Store data locally ✓ Work offline ✓ Perform CRUD operations ✓ Build powerful local database apps in Flutter

You can now edit this documentation and expand it with: - Relationships (Foreign Keys) - Batch Transactions - Clean Architecture integration - Provider / Bloc integration